



**SWAMINARAYAN
UNIVERSITY**
THE RESERVOIR OF ALL KNOWLEDGE

**Faculty of Engineering (Degree)
Semester – 7th**

Subject Name – Information security

Subject Code – BE103707

Faculty Name – Radhika Gokani

CHAPTER -6

1. Message Authentication Codes

- A **Message Authentication Code (MAC)** is a cryptographic technique that ensures **data integrity** and **authentication** using a secret shared key. It's efficient and widely used in secure communication protocols like SSL/TLS, IPsec, and SSH.
- A **Message Authentication Code (MAC)** is a short piece of information (a fixed-size tag) generated from a message and a secret key.
- It is used to **ensure the integrity** (message not altered) and **authenticity** (comes from the right sender) of a message.
- Unlike encryption, MACs do not hide the content of the message—they only protect against tampering and forgery.

◆ **How MAC Works**

1. Sender Side:

- The sender inputs the **message** and a **shared secret key** into a MAC algorithm.
- The algorithm produces a **MAC value (tag)**.
- The message + tag is sent to the receiver.

2. Receiver Side:

- The receiver takes the received **message** and the same **shared secret key**.
- The receiver runs the MAC algorithm on the message.
- If the calculated tag matches the received tag → message is authentic and untampered.

◆ **Properties of MAC**

1. **Integrity** → Detects if the message was modified.
2. **Authentication** → Confirms that the message came from a legitimate sender who knows the secret key.
3. **Non-repudiation (limited)** → Sender cannot easily deny creating the message (since only they know the key).

◆ **Common MAC Algorithms**

- **HMAC (Hash-based Message Authentication Code):**
 - Uses a cryptographic hash function (like SHA-256) + secret key.
 - Widely used (e.g., TLS, IPsec).
- **CMAC (Cipher-based MAC):**
 - Uses block cipher algorithms like AES.

- **UMAC/VMAC:**
 - Designed for high performance.

◆ Difference: MAC vs. Digital Signature

Feature	MAC	Digital Signature
Key Type	Symmetric (shared secret)	Asymmetric (public/private key)
Integrity	✓	✓
Authentication	✓	✓
Non-repudiation	✗ (not strong)	✓
Speed	Faster	Slower

◆ Example (HMAC)

Suppose Alice and Bob share a secret key **K** = "1234".

Message = "HELLO"

HMAC-SHA256(Message, Key) → abc123ef56... (a fixed-size tag)

- Alice sends "HELLO" + **tag** to Bob.
- Bob recomputes HMAC using key = "1234". If tags match, the message is authentic.

2. Its requirements and security

Requirements of MAC

For a MAC to be secure and useful, it must meet certain **functional** and **security** requirements:

1. Functional Requirements

- **Efficiency:**
The MAC algorithm should be fast to compute for both sender and receiver.
- **Fixed Output Size:**
Regardless of message length, the MAC should generate a fixed-size tag (e.g., 128 or 256 bits).
- **Deterministic:**
For the same key and message, the MAC must always produce the same tag.
- **Key Dependence:**
Without knowing the secret key, it should be infeasible to generate a valid MAC for any message.
- **Ease of Verification:**
The receiver should be able to quickly verify authenticity using the same secret key.

2. Security Requirements

- **Resistance to Forgery:**
An attacker should not be able to generate a valid MAC without the key.
- **Collision Resistance (in practice):**
It should be computationally infeasible to find two different messages that produce the same MAC tag under the same key.
- **Replay Protection (context-dependent):**
MACs may need to be combined with sequence numbers/timestamps to prevent attackers from reusing old valid messages.
- **Key Confidentiality:**
The security of the MAC depends entirely on keeping the secret key safe.

Security of MAC

The **security of MACs** relies on the assumption that the secret key is known only to sender and receiver.

Attacks MACs Must Resist

1. **Brute-Force Attack:**
 - Attacker tries all possible keys.
 - Countermeasure → Use sufficiently long keys (128 bits or more).
2. **Birthday on hash-based MACs:**
 - Attacker tries to find collisions in the underlying hash function.
 - Countermeasure **Attack** (→ Use strong hash functions like SHA-256, SHA-3).
3. **Replay Attack:**
 - Re-sending a previously valid message + MAC.
 - Countermeasure → Add sequence numbers, nonces, or timestamps.
4. **Extension Attack (specific to some hash functions):**
 - If the MAC is just a plain hash(message || key), attackers might extend it.
 - Countermeasure → Use **HMAC**, which is designed to resist such attacks.

Example: Why HMAC is Secure

HMAC adds the key in a more complex way:

- $\text{HMAC}(\text{Key}, \text{Message}) = \text{Hash}[(\text{Key} \oplus \text{opad}) \parallel \text{Hash}[(\text{Key} \oplus \text{ipad}) \parallel \text{Message}]]$
This double hashing prevents extension attacks and ensures strong security even if the hash function has weaknesses.

Note:

- **Requirements:** Efficient, key-dependent, fixed-length output, secure verification.
- **Security:** Protects against forgery, collisions, replay, and brute-force.
- **Best practice:** Use modern algorithms (e.g., HMAC-SHA256 or CMAC-AES) with strong keys and additional replay protection mechanisms.

3. MACs based on Hash Functions

MACs Based on Hash Functions

A **hash-based MAC (HMAC)** is a type of Message Authentication Code that uses a **cryptographic hash function** (like SHA-256 or SHA-3) along with a **secret key**.

It was designed to overcome weaknesses of simply doing $\text{MAC} = \text{Hash}(\text{Key} \parallel \text{Message})$.

Why Not Just Hash?

If we use:

- $\text{MAC} = \text{Hash}(\text{Key} \parallel \text{Message})$
- or
- $\text{MAC} = \text{Hash}(\text{Message} \parallel \text{Key})$

then certain **attacks** (like *length extension attacks*) can allow an attacker to forge valid MACs, especially when using Merkle–Damgård hash functions (MD5, SHA-1, SHA-256).

The HMAC Construction

HMAC solves this by mixing the key into the hash function in a **nested way**.

Formula:

$$\text{HMAC}(K, M) = H[(K \oplus \text{opad}) \parallel H[(K \oplus \text{ipad}) \parallel M]]$$

$$\text{HMAC}(K, M) = H[(K \oplus \text{opad}) \parallel H[(K \oplus \text{ipad}) \parallel M]]$$

Where:

- **H** = underlying hash function (e.g., SHA-256)
- **K** = secret key (padded/truncated to block size)
- **M** = message
- **opad** = outer padding (0x5c repeated)
- **ipad** = inner padding (0x36 repeated)

◆ Properties of HMAC

1. **Key-dependent:** Without the secret key, generating a valid HMAC is infeasible.
2. **Collision resistance:** Security does not directly depend on collisions of H, but on the difficulty of forging the tag.
3. **Proven Security:** HMAC security can be reduced to the security of the underlying hash function.
4. **Widely used:** Found in TLS, IPsec, SSL, SSH, JWT authentication, etc.

◆ Example of HMAC

Suppose:

- Message = "HELLO"
- Key = "secret123"
- Hash function = SHA-256

Then:

$\text{HMACSHA256}(\text{"secret123"}, \text{"HELLO"}) = 3615\text{f}80\text{c}9\text{d} \dots (256\text{-bit tag})$
 $\text{HMAC}_{\{\text{SHA256}\}}(\text{"secret123"}, \text{"HELLO"}) = 3615\text{f}80\text{c}9\text{d} \dots (256\text{-bit tag})$
 $\text{HMACSHA256}(\text{"secret123"}, \text{"HELLO"}) = 3615\text{f}80\text{c}9\text{d} \dots (256\text{-bit tag})$

◆ Advantages of Hash-Based MACs

- **Efficiency:** Faster than cipher-based MACs (like CMAC) in software.
- **Flexibility:** Works with different hash functions (SHA-256, SHA-512, SHA-3).
- **Security:** Resistant to length-extension attacks (unlike plain Hash(key||msg)).

Note:

- MACs based on hash functions are called **HMACs**.
- They use a **nested hashing construction** with inner and outer padding.
- They are efficient, secure, and widely deployed in real-world security protocols.

4. Macs based on Block Ciphers

- **CBC-MAC** → simple, secure only for fixed-length messages.
- **CMAC** → improved, secure for variable-length messages, standardized with AES.
- Block-cipher-based MACs are widely used in **embedded systems, smart cards, and hardware implementations** because AES is efficient in hardware.

MACs Based on Block Ciphers

Instead of using a hash function, these MACs use a **symmetric block cipher** (like DES, AES) with a **secret key** to generate a tag.

The most common ones are:

1. **CBC-MAC** (Cipher Block Chaining MAC)
 2. **CMAC** (Cipher-based MAC, an improved version of CBC-MAC)
- #### 1. CBC-MAC (Cipher Block Chaining MAC)
- Works by encrypting the message in **CBC mode** with a block cipher (e.g., AES).
 - The **last ciphertext block** is taken as the MAC tag.

Process:

1. Break message into blocks $M_1, M_2, \dots, M_n$
2. Encrypt using CBC mode with key K.
3. Output last block = MAC.

Formula:

$\text{MAC}_K(M) = \text{Cn}(\text{last ciphertext block})$
 $\text{MAC}_K(M) = \text{Cn} \quad \text{last ciphertext block}$

block))MACK(M)=Cn(last ciphertext block)

Advantages:

- Simple and efficient.
- Strong security for fixed-length messages.

Limitations:

- Not secure for variable-length messages (can be attacked if attacker appends data).
- Requires message padding.

2. CMAC (Cipher-based MAC)

- An **improved version of CBC-MAC**, standardized by NIST.
- Designed to fix weaknesses of CBC-MAC for variable-length messages.
- Uses two subkeys K_1, K_2 , K_1, K_2 derived from the main key and applies them depending on whether the last block is complete or padded.

Advantages over CBC-MAC:

- Secure for variable-length messages.
- Standardized and widely used (AES-CMAC).

💡 **Security of Block-Cipher-based MACs**

- Security relies on the **strength of the block cipher** (AES is recommended, DES is outdated).
- Resistant to forgery without the secret key.
- Vulnerable if used incorrectly (e.g., CBC-MAC without length-control).

💡 **Comparison: Hash-based vs. Block Cipher-based MACs**

Feature	Hash-based MAC (HMAC)	Block Cipher-based MAC (CBC-MAC / CMAC)
Underlying Tool	Hash function (SHA-256, SHA-3)	Block cipher (AES, DES)
Performance	Faster in software	Faster in hardware
Message Length	Works for variable length	CBC-MAC only safe for fixed length, CMAC fixes this
Security Basis	Hash function strength	Block cipher strength
Popular Standard	HMAC (SHA-256)	CMAC (AES-CMAC)
